

0.94. Un jeu de bombes, avec des briques — des briques M

Le jeu du 0.93.9, et des briques M, posées au hasard dans les couloirs, une case sur trois environ. Elles bloquent le O, les W et les flammes, comme les X. Le coin du départ reste libre.

1. Ce qu'il faut comprendre

C'est le jeu du 0.93.9, avec une chose en plus : des briques. Des M remplissent une partie des couloirs, au hasard : quand on rejoue, le terrain change. Pour l'instant, elles sont aussi dures que les X ; au 0.94.1, les bombes les casseront.

Ce qui est nouveau ici : poser les briques au hasard. Deux boucles passent sur l'intérieur du terrain (lignes 1 à 15, colonnes 1 à 17), comme pour les piliers (le 0.93). Une brique est posée si **trois** conditions sont vraies à la fois (`&&`, le 0.75).

1. C'est un couloir : `c % 2 == 1 || 1 % 2 == 1`. Un pilier a sa colonne **et** sa ligne paires ; un couloir a sa colonne **ou** sa ligne impaire (`% 2 == 1`). Les **parenthèses** autour de cette condition la calculent d'abord, comme au 0.91.6 : « couloir » ET le reste.

2. Ce n'est pas le coin du départ : `c + 1 > 5`. En (1, 1), $1 + 1 = 2$; en (3, 1), 4 ; en (3, 2), 5 ; en (1, 4), 5 : jamais de brique. En (5, 1), 6 : une brique possible. Ce coin libre, c'est de la place pour poser sa première bombe et s'abriter : de (1, 1), on peut aller en (3, 2), hors des bras de la flamme.

3. Une fois sur trois : `hasard() % 3 == 0`. Le reste par 3 vaut 0, 1 ou 2 : il vaut 0 une fois sur trois environ. Exemple : `hasard()` rend 201, $201 = 67 \times 3 + 0$: une brique. 202 : reste 1, pas de brique.

Les briques sont posées AVANT le O et les W. Si le hasard met un M dans le coin d'un W, le W est posé par-dessus : il est là, mais enfermé jusqu'à ce qu'une bombe le libère.

Le M bloque tout le monde : les W ne vont que sur une case vide (le 0.93.5) ; le O aussi (le 0.93.2). Mais attention au 0.93.8 : quand le O essayait d'aller sur une case « ni vide, ni X, ni B », c'était un W ou une flamme, et il **perdait**. Un M n'est ni l'un ni l'autre : on ajoute `&& lire(nx, ny) != ALPHABET[12]`, sinon toucher une brique ferait perdre.

Le M arrête les flammes, comme un X : dans `flammes`, `if (lire(...) == ALPHABET[23] || lire(...) == ALPHABET[12]) break;` (`||`, « ou »). Une brique **protège** ce qu'il y a derrière elle.

Le jeu devient plus dur : les W sont souvent enfermés, et il faut faire son chemin entre les briques. Au 0.94.1, on pourra le **creuser** avec les bombes.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <hasard>` → [0.81.2. La fonction hasard\(\) — un nombre au hasard](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2843 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	108 o	← la console, à l'allumage — la mise en route de la console
chaque_minuteur()	124 o	← main() — une fonction fournie par le langage
toucheW()	81 o	← caseFeu() — ta fonction
caseFeu()	120 o	← flammes() — ta fonction
flammes()	373 o	← main() — ta fonction
viderEcran()	33 o	← nouvellePartie(), finPartie() — ta fonction
nouvellePartie()	337 o	← main() — ta fonction
finPartie()	51 o	← main() — ta fonction
main()	774 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← effacer(), lire(), texte(), poser() — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() — efface des cases
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
Reste	10 o	← nouvellePartie() — le reste d'une division
CopierTuiles	94 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Hasard	25 o	← hasard() — tire un nombre au hasard
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	496 o	← CopierTuiles — 31 lettres sur 43
" "	1 o	← texte() — un texte gravé
"_"	1 o	← texte() — un texte gravé
"SCORE"	5 o	← texte() — un texte gravé
"FIN"	3 o	← texte() — un texte gravé
"SCORE"	5 o	← texte() — un texte gravé
"START : REJOUER"	15 o	← texte() — un texte gravé
"BRAVO !"	7 o	← texte() — un texte gravé

Lettres gravées : ABCEFIJMNORSTUVWX0123456789!-:|

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

lire()	× 14	202 o sur place, dans caseFeu(), flammes(), main() + AdresseCase (10 o)
poser()	× 10	178 o sur place, dans nouvellePartie(), main() + AdresseCase (10 o)
texte()	× 7	91 o sur place, dans caseFeu(), nouvellePartie(), finPartie(), main() + AdresseCase (10 o)

```

o), EcrireTexte (10 o)
  effacer()      x 4    75 o sur place, dans caseFeu(), viderEcran(), main() + AdresseCase (10 o), EffacerCases
(12 o)
  bouton()       x 6    30 o sur place, dans main() + LireManette (37 o)
  nombre()       x 2    20 o sur place, dans finPartie(), main() + EcrireNombre (33 o)
  hasard()       x 2     6 o sur place, dans nouvellePartie(), main() + Hasard (25 o)
  image()        x 1     3 o sur place, dans main() + AttendreImage (28 o)
  images()       x 1     2 o sur place, dans chaque_minuteur()
Tes fonctions appelées :
  caseFeu()      x 5   108 o sur place, dans flammes()
  chaque_minuteur() x 2   21 o sur place, dans main()
  flammes()      x 2   13 o sur place, dans main()
  toucheW()     x 1   11 o sur place, dans caseFeu()
  viderEcran()  x 2     6 o sur place, dans nouvellePartie(), finPartie()
  nouvellePartie() x 2     6 o sur place, dans main()
  finPartie()   x 2     6 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// ---- UN JEU DE BOMBES ----
// Les lettres : X = un mur, O = le joueur, B = une bombe, W = un ennemi,
// M = une brique (NOUVEAU).
// ALPHABET[23] : le X    ALPHABET[14] : le O
// ALPHABET[1]  : le B    ALPHABET[22] : le W
// ALPHABET[12] : le M

#include <effacer>    // efface des cases, ou tout le fond
#include <lire>       // lit la tuile posée sur une case
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <texte>      // écrit un texte à l'écran
#include <poser>      // pose une tuile sur une case du fond
#include <hasard>     // tire un nombre au hasard
#include <nombre>     // écrit un nombre en chiffres
#include <chaque>     // répond 1 toutes les n millisecondes
#include <bouton>     // lit un bouton de la manette
#include <reste>      // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair

```

2. UNE VARIABLE — OX

```
uint8_t ox = 1;          // le O, le joueur : sa colonne... (le 0.93.1)
```

3. UNE VARIABLE — OY

```
uint8_t oy = 1;          // ...et sa ligne. Il part du coin (1, 1).
```

4. UNE VARIABLE — NX

```
uint8_t nx = 0;          // la case d'ARRIVÉE, calculée AVANT de bouger (le 0.82)
```

5. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

6. UNE VARIABLE — BOMBE

```
uint8_t bombe = 0;       // 1 : une bombe est posée ; 0 : pas de bombe (le 0.93.2)
```

7. UNE VARIABLE — BX

```
uint8_t bx = 0;          // la place de la bombe : sa colonne...
```

8. UNE VARIABLE — BY

```
uint8_t by = 0;           // ...et sa ligne
```

9. UNE VARIABLE — BDELAI

```
uint8_t bdelai = 0;       // les images passées depuis la pose de la bombe (le 0.93.3)
```

10. UNE VARIABLE — FEU

```
uint8_t feu = 0;         // 1 : les flammes sont à l'écran ; 0 : non
```

11. UNE VARIABLE — FDELAI

```
uint8_t fdelai = 0;      // les images passées depuis l'explosion
```

12. UNE VARIABLE — FX

```
uint8_t fx = 0;          // le centre de l'explosion : sa colonne...
```

13. UNE VARIABLE — FY

```
uint8_t fy = 0;          // ...et sa ligne
```

14. UN TABLEAU — EX

```
uint8_t ex[3];           // les TROIS W : leurs colonnes... (le 0.93.6)
```

15. UN TABLEAU — EY

```
uint8_t ey[3];           // ...et leurs lignes. Le W n° i est en (ex[i], ey[i]).
```

16. UNE VARIABLE — D

```
uint8_t d = 0;           // la direction tirée au hasard, 0 à 3 (le 0.93.5)
```

17. UN TABLEAU — VIVANT

```
uint8_t vivant[3];       // vivant[i] : 1 si le W n° i est là, 0 s'il est détruit (le 0.93.7)
```

18. UNE VARIABLE — SCORE

```
uint8_t score = 0;       // les W détruits
```

19. UNE VARIABLE — ECRAN

```
uint8_t ecran = 0;       // 0 = le JEU, 1 = la FIN (le 0.93.8)
```

20. UNE VARIABLE — MORT

```
uint8_t mort = 0;        // 1 : le 0 vient d'être touché

// ---- une flamme sur la case (c, l) : y a-t-il un W ? (le 0.93.7) ----
// On cherche le W n° i qui est vivant ET sur cette case. S'il y en a un,
// il est détruit (vivant[i] = 0) et le score gagne 1.
```

21. UNE FONCTION — TOUCHEW()

```
void toucheW(uint8_t c, uint8_t l) {
    for (uint8_t i = 0; i < 3; i++) {
        if (vivant[i] == 1 && ex[i] == c && ey[i] == l) {
            vivant[i] = 0;
            score = score + 1;
        }
    }
}

// ---- UNE case de flamme (le 0.93.3) ----
// allume = 1 : on dessine la flamme ; allume = 0 : on l'efface.
// debout = 1 : une flamme verticale, le | ; debout = 0 : horizontale, le -.
```

22. UNE FONCTION — CASEFEU()

```
void caseFeu(uint8_t c, uint8_t l, uint8_t allume, uint8_t debout) {
    if (allume == 0) {
        effacer(c, l, 1); // la flamme s'éteint : la case est vide
    } else {
        if (lire(c, l) == ALPHABET[22]) { // un W sous la flamme ? (le 0.93.7)
            toucheW(c, l);
        }
        if (c == ox && l == oy) { // le 0 sous la flamme : perdu (le 0.93.8)
            mort = 1;
        }
        if (debout == 1) {
            texte(c, l, "|");
        } else {
            texte(c, l, "-");
        }
    }
}

// ---- Toute l'explosion (le 0.93.3) : des bras de DEUX cases (le 0.93.4) ----
//
//      |
//      |      chaque bras : jusqu'à 2 cases,
//      |      mais il s'arrête au premier X.
//  - - - - -
//      |      break : on sort de la boucle, tout de suite.
//      |
```

23. UNE FONCTION — FLAMMES()

```
void flammes(uint8_t allume) {
    caseFeu(fx, fy, allume, 0);           // le centre
    for (uint8_t k = 1; k <= 2; k++) {    // à droite : fx + 1, puis fx + 2
        if (lire(fx + k, fy) == ALPHABET[23] || lire(fx + k, fy) == ALPHABET[12]) break; // un X ou un M (NOUVEAU)
    : le bras s'arrête
        caseFeu(fx + k, fy, allume, 0);
    }
    for (uint8_t k = 1; k <= 2; k++) {    // à gauche : fx - 1, puis fx - 2
        if (lire(fx - k, fy) == ALPHABET[23] || lire(fx - k, fy) == ALPHABET[12]) break;
        caseFeu(fx - k, fy, allume, 0);
    }
    for (uint8_t k = 1; k <= 2; k++) {    // en bas
        if (lire(fx, fy + k) == ALPHABET[23] || lire(fx, fy + k) == ALPHABET[12]) break;
        caseFeu(fx, fy + k, allume, 1);
    }
    for (uint8_t k = 1; k <= 2; k++) {    // en haut
        if (lire(fx, fy - k) == ALPHABET[23] || lire(fx, fy - k) == ALPHABET[12]) break;
        caseFeu(fx, fy - k, allume, 1);
    }
}

// ---- vider l'écran, comme dans le serpent (le 0.93.8) ----
```

24. UNE FONCTION — VIDERECRAN()

```
void viderEcran() {
    for (uint8_t l = 0; l < 18; l++) {
        effacer(0, l, 20);
    }
}

// ---- une partie qui commence, TOUT au départ (le 0.93.8) ----
// C'était le début de main() ; c'est maintenant une fonction, pour pouvoir
// la rappeler quand on rejoue.
```

25. UNE FONCTION — NOUVELLEPARTIE()

```
void nouvellePartie() {
    ox = 1;                                // le 0 revient dans son coin
    oy = 1;
    bombe = 0;                             // pas de bombe
    feu = 0;                               // pas de flammes
    score = 0;
    mort = 0;
    ecran = 0;                             // on est sur le JEU
    viderEcran();

    // ---- Le terrain : le cadre et les piliers X (le 0.93) ----
    for (uint8_t l = 0; l < 17; l++) {
        for (uint8_t c = 0; c < 19; c++) {
            if (l == 0 || l == 16 || c == 0 || c == 18) { // le cadre
                poser(c, l, ALPHABET[23]);
            }
            if (c % 2 == 0 && l % 2 == 0) {                // les piliers : colonne ET ligne paires
                poser(c, l, ALPHABET[23]);
            }
        }
    }

    // ---- NOUVEAU : les briques M, au hasard, dans les couloirs ----
    // Une case est un couloir si sa colonne OU sa ligne est impaire
    // (% 2 == 1) : ce n'est pas un pilier (le 0.93).
    // c + 1 > 5 : le coin du départ reste libre. (1, 1), (2, 1), (3, 1),
    // (4, 1), (1, 2), (3, 2), (1, 3)... ont c + 1 <= 5 : jamais de brique.
    // Sans ça, le 0 pourrait être enfermé, et sa première bombe le tuerait.
    // hasard() % 3 == 0 : le reste par 3 vaut 0, 1 ou 2 ; 0 une fois sur trois.
    for (uint8_t l = 1; l < 16; l++) {
        for (uint8_t c = 1; c < 18; c++) {
            if ((c % 2 == 1 || l % 2 == 1) && c + 1 > 5 && hasard() % 3 == 0) {
                poser(c, l, ALPHABET[12]);
            }
        }
    }

    poser(ox, oy, ALPHABET[14]);            // le 0 à sa place (le 0.93)

    // les trois W, dans trois coins (le 0.93.6)
    ex[0] = 17;                             // le n° 0 : en bas à droite
    ey[0] = 15;
    ex[1] = 17;                             // le n° 1 : en haut à droite
    ey[1] = 1;
    ex[2] = 1;                             // le n° 2 : en bas à gauche
    ey[2] = 15;
    for (uint8_t i = 0; i < 3; i++) {
        vivant[i] = 1;                       // au départ, les trois sont vivants
        poser(ex[i], ey[i], ALPHABET[22]);
    }
    texte(0, 17, "SCORE");                  // le score, sous le terrain (le 0.93.7)
}

// ---- la partie s'arrête : l'écran FIN (le 0.93.8) ----
```

26. UNE FONCTION — FINPARTIE()

```
void finPartie() {  
    ecran = 1; // on est sur la FIN  
    viderEcran();  
    texte(8, 6, "FIN");  
    texte(5, 9, "SCORE");  
    nombre(11, 9, score);  
    texte(2, 13, "START : REJOUER");  
}
```


27. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    nouvellePartie();                // la première partie (le 0.93.8)

    while (true) {
        image();

        if (ecran == 0) {            // le JEU (le 0.93.8)
            // ---- le 0 bouge avec la croix, un pas toutes les 150 ms (le 0.93.1) ----
            if (chaque(150)) {
                // 1. la case d'ARRIVÉE : on part de la case actuelle (le 0.82).
                //     else if : une seule flèche à la fois, pas de pas en diagonale.
                nx = ox;
                ny = oy;
                if (bouton(DROITE)) {
                    nx = ox + 1;
                } else if (bouton(GAUCHE)) {
                    nx = ox - 1;
                } else if (bouton(BAS)) {
                    ny = oy + 1;
                } else if (bouton(HAUT)) {
                    ny = oy - 1;
                }

                // 2. une flèche est tenue (l'arrivée n'est pas la case où l'on est)...
                if (nx != ox || ny != oy) {
                    // 3. l'arrivée est VIDE (le 0.93.2) : le 0 y va.
                    if (lire(nx, ny) == 0) {
                        effacer(ox, oy, 1);                // le 0 quitte sa case...
                        if (bombe == 1 && ox == bx && oy == by) {
                            poser(bx, by, ALPHABET[1]);    // ...mais s'il était sur la bombe, la B apparaît (le 0.93.2)
                        }
                        ox = nx;                            // ...et va sur la case d'arrivée
                        oy = ny;
                        poser(ox, oy, ALPHABET[14]);
                    } else if (lire(nx, ny) != ALPHABET[23] && lire(nx, ny) != ALPHABET[1] && lire(nx, ny) !=
ALPHABET[12]) {
                        mort = 1;                            // ni vide, ni X, ni B, ni M (NOUVEAU) : un W ou une flamme ! (le
0.93.8)
                    }
                }
            }

            // ---- A : poser une bombe, là où est le 0 (le 0.93.2) ----
            // bombe == 0 : une seule bombe à la fois.
            // feu == 0 : pas pendant les flammes (le 0.93.3).
            if (bouton(A) && bombe == 0 && feu == 0) {
                bombe = 1;
                bx = ox;                                // la bombe est sous le 0
                by = oy;
                bdelai = 0;                            // on commence à compter (le 0.93.3)
            }

            // ---- la bombe explose au bout de 2 secondes (le 0.93.3) ----
            // image() revient 60 fois par seconde : 120 images, 2 secondes.
            if (bombe == 1) {
                bdelai = bdelai + 1;
                if (bdelai == 120) {
                    bombe = 0;                            // plus de bombe...
                    fx = bx;                                // ...elle explose, là où elle était
                    fy = by;
                    feu = 1;
                    fdelai = 0;
                    flammes(1);                            // on dessine les flammes
                }
            }
        }
    }
}
```

```

}

// ---- les flammes s'éteignent au bout d'une demi-seconde (30 images) (le 0.93.3) ----
if (feu == 1) {
    fdelai = fdelai + 1;
    if (fdelai == 30) {
        feu = 0;
        flammes(0); // on efface les MÊMES cases
        poser(ox, oy, ALPHABET[14]); // le 0 était peut-être dans une flamme : on le remet
        for (uint8_t i = 0; i < 3; i++) { // et les W encore vivants
            if (vivant[i] == 1) {
                poser(ex[i], ey[i], ALPHABET[22]);
            }
        }
    }
}

// ---- les W se promènent au hasard (le 0.93.5) : les TROIS, un par un (le 0.93.6) ----
if (chaque(400)) {
    for (uint8_t i = 0; i < 3; i++) {
        if (vivant[i] == 1) { // un W détruit ne bouge plus (le 0.93.7)
            d = hasard() % 4; // sa direction, au hasard (le 0.93.5)
            nx = ex[i];
            ny = ey[i];
            if (d == 0) nx = ex[i] + 1;
            if (d == 1) ny = ey[i] + 1;
            if (d == 2) nx = ex[i] - 1;
            if (d == 3) ny = ey[i] - 1;
            if (nx == ox && ny == oy) {
                mort = 1; // le W arrive sur le 0 : perdu (le 0.93.8)
            }
            if (lire(nx, ny) == 0) { // seulement sur une case VIDE
                effacer(ex[i], ey[i], 1);
                ex[i] = nx;
                ey[i] = ny;
                poser(ex[i], ey[i], ALPHABET[22]);
            }
        }
    }
}

nombre(6, 17, score); // le score, à chaque image (le 0.93.7)

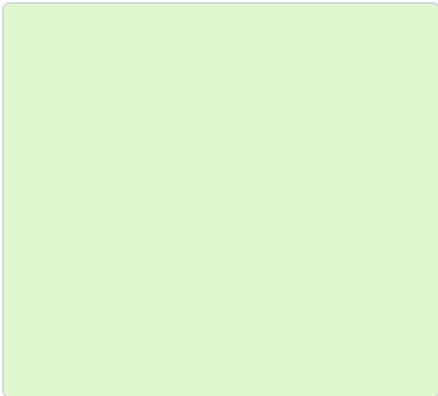
// ---- touché (par un W ou une flamme) : la partie s'arrête (le 0.93.8) ----
if (mort == 1) {
    finPartie();
} else if (score == 3) {
    // ---- les trois W détruits : GAGNÉ (le 0.93.9) ----
    finPartie(); // le même écran FIN, avec le score...
    texte(6, 3, "BRAVO !"); // ...et BRAVO au-dessus
}

// ---- sur la FIN, START : on rejoue (le 0.93.8) ----
if (ecran == 1 && bouton(START)) {
    nouvellePartie();
}
}
}

```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran n'a pas bougé



DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran n'a pas bougé



START — l'écran a changé

5. Ce qu'on doit voir

Des briques M au hasard dans les couloirs ; le coin du départ est libre ; ni le O, ni les W, ni les flammes ne passent.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ des briques M dans le terrain (55 M)
- ✓ jamais sur un pilier ni dans le coin du départ
- ✓ à droite, le O s'arrête devant un M ou un X, sans perdre (devant lui : « M »)

Ensuite : **0.94.1. Un jeu de bombes, avec des briques — la flamme casse les briques** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++